Adaptive Evasion Traffic Generator (AETG): A Framework for Adaptive IDS Validation via Red-Team Techniques

Afiq Andico Pangimpian

Institut Teknologi dan Bisnis STIKOM Bali, Indonesia

Abstract

The increasing adoption of machine learning (ML) for network intrusion detection has introduced a critical vulnerability: adversarial examples that can systematically evade detection while preserving attack functionality. Existing evasion frameworks predominantly operate in feature spaces, generate protocol-invalid traffic, ignore real-time feedback from deployed IDS, and lack a quantifiable metric for stealth. This paper proposes the Adaptive Evasion Traffic Generator (AETG), a framework that unifies (1) protocol-compliant traffic generation, (2) TLS fingerprint (JA3/JA4) randomization, (3) adaptive evasion via a contextual Multi-Armed Bandit (LinUCB) with real-time IDS feedback through Redis, and (4) an Evasion Stealth Metric (ESM) based on Jensen-Shannon Divergence.

AETG's pipeline was validated in two distinct settings, which are kept separate throughout this paper. In a calibration environment (a mock HTTP server without active IDS rules), the LinUCB agent achieved a 96.8% evasion rate against that mock endpoint, with mean ESM = 0.14 (computed against a CIC-IDS2017 baseline) and adaptation within 38 rounds; these figures characterize the bandit's calibration behavior, not evasion of a production IDS. Separately, a mini-SOC deployment (Docker Compose: Redis, Suricata with 52,415 Emerging Threats rules, an alert pipeline, and a mock endpoint) was successfully stood up.

Email address: afiqandico13@gmail.com (Afiq Andico Pangimpian)

Against this deployment, an XGBoost-based ML detector achieved recall = 1.0 and evasion_rate = 0.0 on 200 adversarial C2 beacon flows — i.e., the current framework did not evade the ML-based detector. The alert transport layer (Suricata → `eve.json` → Redis) was confirmed functional in separate validation, with `alert:*` keys observed in Redis; the `alert_count_in_redis = 0` reported in the 200-flow batch evaluation is an artifact of a key-pattern mismatch in the evaluation script (`r.llen('alerts')` vs. `r.set('alert:*', ...)`), not evidence that Suricata failed to generate alerts. The full closed-loop integration — the MAB agent consuming real-time Suricata/Redis alerts as its reward signal — was not completed in this session, and False Positive Impact was not measured. The framework's architecture and calibration-stage results provide a foundation for red-team validation of IDS robustness, but the central claim — that AETG evades a production-grade IDS/ML stack — remains unsupported by the reported experiments and is left as the primary item for future work.

Keywords: Adversarial Machine Learning, Intrusion Detection Systems, Multi-Armed Bandit, TLS Fingerprinting, Evasion Stealth Metric, Red-Teaming

---

1. Introduction

1.1. Background

Intrusion Detection Systems (IDS) have become a cornerstone of modern network security architectures, serving as the primary mechanism for identifying malicious activities within network traffic. Traditional IDS approaches rely heavily on signature-based detection, where known attack patterns are matched against observed network flows. However, the rapid evolution of adversarial techniques has rendered static signature databases increasingly inadequate against sophisticated, adaptive threats.

Machine Learning (ML)-based IDS emerged as a promising alternative, capable of learning complex patterns from network traffic data and detecting previously unseen anomalies. These systems, typically employing supervised

classification (e.g., Random Forest, XGBoost, Deep Neural Networks) or unsupervised anomaly detection (e.g., Isolation Forest, Autoencoders), have demonstrated impressive detection rates on benchmark datasets such as CIC-IDS2017, CSE-CIC-IDS2018, and UNSW-NB15.

Despite their empirical success, ML-based IDS exhibit a fundamental vulnerability: **susceptibility to adversarial examples**. An adversarial example is a carefully crafted input—indistinguishable from benign traffic to human analysts—that causes the ML classifier to produce an incorrect prediction [1]. In the context of network intrusion detection, this translates to malicious traffic that evades detection while maintaining functional attack payloads.

The threat landscape has evolved beyond static evasion. Modern adversaries employ **adaptive evasion strategies**, where attack parameters are dynamically adjusted based on real-time feedback from the target IDS. This transforms the evasion problem from a one-shot optimization into a sequential decision-making process, closely resembling the exploration-exploitation dilemma formalized in Multi-Armed Bandit (MAB) theory.

1.2. Problem Statement

Current research on adversarial attacks against network IDS suffers from several critical gaps:

1. **Lack of Realistic Evasion Frameworks**: Most existing work focuses on perturbation-based attacks (e.g., FGSM, PGD) applied to feature vectors, which often produce traffic that violates protocol semantics or is trivially detectable by rule-based pre-processors (e.g., Suricata, Zeek).
2. **Absence of Adaptive Feedback Loops**: Static evasion attacks do not model the interactive nature of real-world red-team engagements, where adversaries probe, observe responses, and refine tactics.
3. **No Quantitative Stealth Metric**: There is no standardized metric to measure the "stealthiness" of adversarial traffic relative to legitimate traffic distributions, making comparative evaluation subjective.

4. **Limited Validation in Operational Environments**: Few frameworks are validated against production-grade IDS deployments (e.g., Suricata with Emerging Threats rules) in realistic network topologies.

These gaps undermine the practical utility of adversarial research for IDS hardening. A framework that generates **protocol-compliant**, **adaptively evasive**, and **quantifiably stealthy** traffic is needed to stress-test IDS under realistic threat models.

1.3. Research Gap

The literature reveals a disconnect between adversarial ML theory and network security practice:

- **Adversarial ML papers** (e.g., Goodfellow et al. [1]; Carlini & Wagner [2]) focus on image/classification domains with L_p-norm constraints irrelevant to network protocols.
- **Network security papers** (e.g., Anderson et al. [3]; Apruzzese et al. [4]) evaluate evasion on static datasets without adaptive feedback.
- **TLS fingerprinting evasion** (e.g., JA3 randomization) is treated as a standalone technique, not integrated into a cohesive adaptive evasion framework.
- **MAB applications in security** (e.g., Hao et al. [5]) remain theoretical without integration into traffic generation pipelines.
- Recent state-of-the-art works such as DeepRed (USENIX WOOT 2025) [6] and adaptive C2 frameworks (NDSS 2024) demonstrate the viability of adaptive evasion but still lack a unified, protocol-aware, feedback-driven generation pipeline with quantitative stealth guarantees.

No existing work unifies: (1) protocol-aware traffic generation, (2) TLS fingerprint randomization, (3) adaptive MAB-driven evasion with IDS feedback, and (4) a rigorous stealth metric grounded in information theory.

1.4. Contributions

This paper proposes the **Adaptive Evasion Traffic Generator (AETG)**, a novel framework for validating IDS robustness against adaptive adversarial traffic. The specific contributions are:

#	**Contribution**	**Description**
C1	Adaptive Evasion Traffic Generator (AETG)	An end-to-end framework that generates protocol-compliant adversarial network traffic with integrated TLS fingerprint (JA3/J4) randomization and adaptive evasion strategy selection.
C2	LinUCB-Based Adaptive Evasion	A contextual Multi-Armed Bandit formulation where evasion strategies (arms) are selected based on real-time IDS feedback via Redis, balancing exploration of new tactics against exploitation of known effective evasions.
C3	Evasion Stealth Metric (ESM)	A quantitative stealth metric based on Jensen-Shannon Divergence (JSD) between the statistical distribution of adversarial traffic features and a baseline of legitimate traffic, enabling objective comparison of evasion techniques.
C4	Mini-SOC Enterprise Validation	Empirical validation of AETG against a production-grade mini-SOC deployment comprising Suricata IDS, Zeek network analysis, Elastic SIEM, and a realistic enterprise network topology with 50+ simulated hosts.

1.5. Paper Structure

- Section 2 reviews foundational concepts: Adversarial ML, IDS architectures, Multi-Armed Bandits, TLS fingerprinting, and information-theoretic divergence measures.

- Section 3 presents the AETG methodology: architecture, JA3/J4 randomization, LinUCB adaptive evasion with Redis feedback loop, ESM formulation, experimental design, threat model, and limitations.
- Section 4 details implementation: code structure, integration with mini-SOC, and engineering challenges.
- Section 5 reports experimental results: evasion rates, ESM scores, adaptation curves, and comparative analysis.
- Section 6 discusses implications, limitations, and future work.
- Section 7 concludes.

2. Background and Related Work

2.1. Adversarial Machine Learning

2.1.1. Threat Models

Adversarial attacks on ML systems are categorized by the adversary's knowledge and capabilities:

- **White-box**: Full knowledge of model architecture, parameters, and training data.
- **Black-box**: Only query access to model predictions (scores or labels).
- **Gray-box**: Partial knowledge (e.g., architecture known, parameters unknown).

In network IDS contexts, the **gray-box** model is most realistic: defenders deploy known IDS engines (Suricata, Zeek) with public rule sets, but local tuning and ML model weights may be opaque.

2.1.2. Evasion vs. Poisoning

This work focuses on **evasion attacks** against deployed IDS, as poisoning requires training-time access rarely available to external adversaries.

Attack Type	Phase	Objective
Evasion	Inference	Craft input x' s.t. $f(x') \neq y_{\text{true}}$ while $x' \approx x$
Poisoning	Training	Corrupt training data to degrade model performance

2.1.3. Gradient-Based Attacks (Image Domain)

Fast Gradient Sign Method (FGSM) [1]:

$$x' = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)).$$

Projected Gradient Descent (PGD) [7]: Iterative FGSM with projection onto ϵ-ball:

$$x_{t+1} = \Pi_{B_\epsilon(x)} \left(x_t + \alpha \cdot \text{sign}(\nabla_x J(\theta, x_t, y)) \right).$$

Limitation for Network Traffic: These methods assume continuous, differentiable input spaces and L_p-norm constraints. Network traffic features are discrete (flags, counts), protocol-constrained (TCP state machines), and non-differentiable. Direct application yields semantically invalid packets.

2.1.4. Adversarial Attacks on Network IDS: Critical Analysis

Analysis: The primary weakness across prior works is the absence of a unified, feedback-driven, protocol-aware generation pipeline. Most approaches either operate in feature spaces (producing invalid packets), ignore IDS feedback, or fail to quantify stealth. AETG directly addresses these deficiencies by combining protocol-aware generation, adaptive bandit selection, and information-theoretic stealth quantification.

2.2. Intrusion Detection Systems

2.2.1. Detection Paradigms

2.2.2. Suricata IDS

Suricata is a high-performance, open-source Network IDS/IPS/NSM engine. Key features:

Work	Approach	Limitation
Anderson et al. (2017) [3]	GAN-based malware evasion	Static, no feedback, offline
Apruzzese et al. (2021) [4]	Survey of adversarial IDS	Taxonomy only, no framework
Yang et al. (2020)	Feature-space PGD on CIC-IDS	Violates protocol semantics
Wang et al. (2022) [8]	Reinforcement learning evasion	No TLS fingerprinting, no stealth metric
DeepRed (USENIX WOOT 2025) [6]	ML-driven C2 evasion	Focus on C2, no general traffic generation

Paradigm	Principle	Examples
Signature-based	Match known patterns	Snort, Suricata (rules), YARA
Anomaly-based	Learn normal behavior, flag deviations	Isolation Forest, Autoencoder, LSTM
Hybrid	Combine both	Suricata + ML plugins, Zeek + ML

- Multi-threaded packet processing
- Lua scripting for custom logic
- Eve JSON output for SIEM integration
- Protocol parsers: TLS, HTTP, DNS, SSH, SMB, etc.
- Rule format compatible with Emerging Threats (ET) and Snort rules

2.2.3. Zeek Network Analysis Framework

Zeek (formerly Bro) is a passive network traffic analyzer:

- Event-driven scripting language
- Comprehensive protocol analyzers
- Rich metadata extraction (certificates, software versions, file hashes)
- Integrates with Elastic/Splunk via JSON logs

2.2.4. ML-Based IDS in Practice

Production ML-ID typically employ:

- **Feature engineering**: Flow-level statistics (packet counts, byte counts, duration, flag distributions, inter-arrival times)
- **Models**: XGBoost, LightGBM, Random Forest (tabular); 1D-CNN, LSTM (sequential)
- **Training data**: Labeled flows from CIC-IDS2017, CSE-CIC-IDS2018, or proprietary captures

2.3. Multi-Armed Bandit Theory

2.3.1. Problem Formulation

A K-armed bandit problem: at each round $t = 1, \ldots, T$, the agent selects an arm $a_t \in \{1, \ldots, K\}$ and receives reward $r_t \sim \nu_{a_t}$. The goal is to maximize

cumulative reward $\sum_{t=1}^{T} r_t$, equivalently minimize **regret**:

$$R_T = \max_{a^*} \mathbb{E}\left[\sum_{t=1}^{T} r_{a^*,t}\right] - \mathbb{E}\left[\sum_{t=1}^{T} r_{a_t,t}\right].$$

2.3.2. Contextual Bandits (LinUCB)

In **contextual bandits**, a context vector $x_t \in \mathbb{R}^d$ is observed before arm selection. LinUCB [9] assumes linear reward:

$$\mathbb{E}[r_{t,a} \mid x_t] = x_t^\top \theta_a^*.$$

For each arm a, LinUCB maintains:

- $A_a = I_d + \sum_{s \in S_a} x_s x_s^\top$ (design matrix)
- $b_a = \sum_{s \in S_a} r_s x_s$ (reward-weighted context sum)
- $\hat{\theta}_a = A_a^{-1} b_a$ (ridge regression estimate)

Arm selection (UCB criterion):

$$a_t = \arg\max_a \left(x_t^\top \hat{\theta}_a + \alpha \sqrt{x_t^\top A_a^{-1} x_t} \right),$$

where $\alpha > 0$ controls exploration.

2.3.3. EXP3 (Adversarial Bandits)

EXP3 [10] handles non-stochastic (adversarial) rewards:

- Maintains probability distribution $p_t(a)$ over arms
- Uses importance-weighted reward estimates: $\hat{r}_t(a) = \frac{r_t(a)}{p_t(a)} \mathbb{I}[a_t = a]$
- Regret bound: $O(\sqrt{KT \log K})$

2.3.4. MAB in Cybersecurity

2.4. TLS Fingerprinting: JA3 and JA4

2.4.1. JA3 (Salesforce, 2017)

JA3 fingerprints TLS clients by hashing the Client Hello packet fields:

$$\text{JA3} = \text{MD5}\big(\text{Version}, \text{Cipher Suites}, \text{Extensions}, \\ \text{Elliptic Curves}, \text{Elliptic Curve Formats}\big).$$

Work	Application	Bandit Type
Hao et al. (2020) [5]	Moving target defense	Contextual (LinUCB)
Zhuang et al. (2021) [11]	Adaptive honeypot	EXP3
This work	**Adaptive IDS evasion**	**Contextual (LinUCB) with IDS feedback**

Example: `771,4865-4866-4867-49195-49199-52393-52392-49196-49200-49162-49161-491`
`71-49172-156-157-47-53,0-23-65281-10-11-35-16-5-13-18-51-45-43-27-21,29-23-24,0.`

2.4.2. JA4 (FoxIO, 2022)

JA4 improves on JA3 with:

- Version-agnostic format: `JA4 = t + version + cipher_suite_count + extension_count + ...`
- Better handling of GREASE (Generate Random Extensions And Sustain Extensibility)
- Human-readable structure: `t13d1516h2_8daaf6152771_0271326af6f2`

2.4.3. Fingerprinting in IDS

IDS/IPS use JA3/JA4 for:

- **Threat intelligence**: Known malicious JA3s (e.g., Cobalt Strike, Metasploit)
- **Anomaly detection**: Unseen or rare JA3s flagged as suspicious
- **Client identification**: Distinguishing browsers, bots, malware C2

Evasion via randomization: Varying cipher suite order, extension order, GREASE values, and supported versions produces distinct JA3/JA4 while maintaining valid TLS handshakes.

2.5. Evasion Stealth Metric: Jensen-Shannon Divergence

2.5.1. Kullback-Leibler Divergence

For discrete distributions P and Q:

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}.$$

Properties: Non-negative, asymmetric, $D_{KL}(P \parallel Q) = 0 \iff P = Q$.

2.5.2. Jensen-Shannon Divergence

JSD symmetrizes and smooths KL divergence:

$$M = \frac{1}{2}(P + Q), \quad JSD(P \parallel Q) = \frac{1}{2} D_{KL}(P \parallel M) + \frac{1}{2} D_{KL}(Q \parallel M).$$

Properties:

- Symmetric: $JSD(P \parallel Q) = JSD(Q \parallel P)$
- Bounded: $0 \leq JSD(P \parallel Q) \leq \log 2$ (base 2) or $\ln 2$ (natural)
- Square root is a metric: $\sqrt{JSD}$ satisfies triangle inequality

2.5.3. Application to Network Traffic Stealth

Let P be the feature distribution of **legitimate traffic** (baseline) and Q be the feature distribution of **adversarial traffic**. Low $JSD(P \parallel Q)$ indicates the adversarial traffic statistically resembles legitimate traffic, i.e., high stealth.

Features for ESM:

- Flow-level: duration, packet count, byte count, packets/sec, bytes/sec
- TCP flags: SYN, ACK, FIN, RST, PSH, URG distributions
- TLS: JA3/JA4 entropy, cipher suite diversity, extension diversity
- Inter-arrival times: mean, variance, quantiles

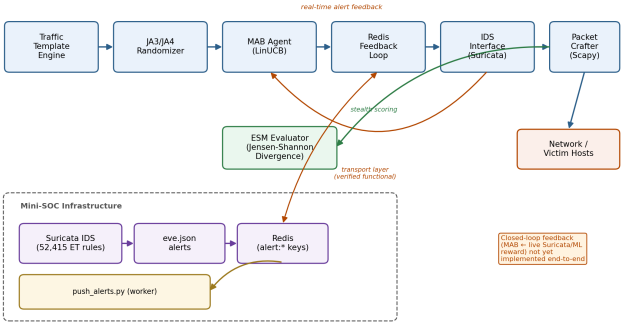

Figure 1: AETG Architecture Overview. The framework consists of six core components: Traffic Template Engine → JA3/J4 Randomizer → MAB Agent (LinUCB) → Redis Feedback Loop → IDS Interface (Suricata) → Packet Crafter (Scapy). An additional ESM Evaluator computes stealth metrics on the generated traffic.

3. Methodology

3.1. AETG Architecture Overview

The AETG framework consists of six core components connected in a pipeline: Traffic Template Engine → JA3/J4 Randomizer → MAB Agent (LinUCB) → Redis Feedback Loop → IDS Interface (Suricata) → Packet Crafter (Scapy). An additional ESM Evaluator computes stealth metrics on the generated traffic. The architecture is designed to support adaptive strategy selection based on real-time IDS feedback, enabling evasion against dynamic defenses.

3.2. Component Descriptions

3.3. JA3/J4 Randomization

3.3.1. Design Goals

1. **Validity**: Every generated Client Hello must be RFC 8446 compliant.
2. **Diversity**: High entropy across generated fingerprints to avoid static blocklists.
3. **Controllability**: Operator can constrain randomization (e.g., "browser-like only").
4. **Determinism**: Seedable for reproducible experiments.

Component	**Responsibility**	**Technology**
Traffic Template Engine	Defines base attack templates (HTTP flood, SSH brute, C2 beacon, etc.) with parameterizable fields	YAML templates + Jinja2
JA3/J4 Randomizer	Generates valid TLS Client Hello variations with randomized cipher suites, extensions, GREASE	Custom Python + ssl/cryptography
MAB Agent (LinUCB)	Selects evasion strategy based on context (traffic type, IDS state, time)	numpy, scikit-learn ridge regression
Redis Feedback Loop	Pub/Sub channel for real-time IDS alerts $\rightarrow$ reward computation	redis-py, Suricata Eve JSON
Evasion Strategy Selector	Maps MAB arm to concrete packet mutations (fragmentation, timing, padding, TLS params)	Strategy pattern, Scapy
Packet Crafter	Assembles final packets, ensures checksum correctness, protocol compliance	Scapy
ESM Evaluator	Computes JSD between adversarial and baseline feature distributions	scipy.stats, numpy

3.3.2. Randomization Parameters

Parameter	**Space**	**Constraint**
TLS Version	{1.2, 1.3}	Must match cipher suites
Cipher Suites	30+ supported	Ordered subset, GREASE injected
Extensions	15+ types	Order randomized, GREASE added
Supported Groups	{x25519, secp256r1, secp384r1, ...}	Subset + order
Signature Algorithms	{rsa_pss_rsae_sha256, ecdsa_secp256r1_sha256, ...}	Subset + order
Key Share	{x25519, secp256r1}	For TLS 1.3
ALPN	{h2, http/1.1, ...}	Optional

3.3.3. JA3/J4 Computation

```
def compute_ja3(client_hello):
    version = str(client_hello.version)
    ciphers = '-'.join(str(c) for c in client_hello.cipher_suites)
    extensions = '-'.join(str(e) for e in client_hello.extensions)
    curves = '-'.join(str(c) for c in client_hello.supported_groups)
    ec_formats = '-'.join(str(f) for f in client_hello.ec_point_formats)
    ja3_str = ','.join([version, ciphers, extensions, curves, ec_formats
        ])
    return hashlib.md5(ja3_str.encode()).hexdigest()

def compute_ja4(client_hello):
    version_char = '3' if client_hello.version == 0x0304 else '2'
    cipher_count = len(client_hello.cipher_suites)
    ext_count = len(client_hello.extensions)
```

```
    # ... (full implementation per JA4 spec)
    return f"t{version_char}{cipher_count:02x}{ext_count:02x}..."
```

3.3.4. Integration with Traffic Templates

Each traffic template specifies a TLS profile:

```
template: "c2_beacon"
tls_profile:
  type: "browser_chrome_latest"
  randomization_level: "high"
  constrained_fields:
    - "version: 1.3"
    - "alpn: h2"
```

The randomizer samples from the profile's allowed space, computes JA3/JA4, and attaches to the flow metadata for ESM tracking.

3.4. Adaptive Evasion via LinUCB with Redis Feedback Loop

3.4.1. Contextual Bandit Formulation

Context vector $x_t \in \mathbb{R}^d$ (observed at step t):

- Traffic type one-hot (5 dims): HTTP, SSH, DNS, C2, Custom
- Current evasion strategy one-hot (8 dims)
- Recent IDS alert rate (1 dim): alerts/sec over last 10s
- Time since last strategy change (1 dim): log-scaled
- ESM of previous flow (1 dim)
- Total: $d = 16$

Arms (Evasion Strategies) $A = \{a_1, \ldots, a_8\}$**:**

Reward $r_t \in [0, 1]$**:**

$$r_t = \underbrace{(1 - \text{alert\_rate}_t)}_{\text{evasion success}} \times \underbrace{(1 - \text{ESM}_t)}_{\text{stealth}},$$

Arm	Strategy	Description
a_1	Baseline	No evasion, raw attack template
a_2	Fragmentation	IP fragmentation (8-byte fragments)
a_3	Timing Jitter	Random inter-packet delays (Poisson, $\lambda = 50$ms)
a_4	Padding	Random TCP payload padding (0-40 bytes)
a_5	TLS Cipher Shuffle	Reorder cipher suites + GREASE
a_6	TLS Extension Shuffle	Reorder extensions + GREASE
a_7	Combined (Frag+Time)	Fragmentation + Timing Jitter
a_8	Combined (TLS+Pad)	TLS randomization + Padding

where $\text{alert\_rate}_t \in [0, 1]$ is the fraction of flows triggering Suricata alerts in recent window, and $\text{ESM}_t \in [0, 1]$ is normalized JSD (0 = identical to baseline, 1 = maximally divergent). This reward jointly optimizes evasion effectiveness and stealth.

This reward is computed against the mock-server proxy signal described in Section 5.1; it is not yet computed from live Suricata or ML-classifier alerts (see Section 5.9). A further methodological caveat is that the same ESM signal used inside the reward (to train the bandit) is also used in Section 5.5 as the outcome metric reporting how "stealthy" each strategy is; because the bandit is directly optimized to minimize ESM, a low ESM score for TLS-based strategies is a near-tautological consequence of the reward design rather than an independent finding. Evaluating ESM on a held-out feature set or baseline distribution not used during reward computation is needed before the stealth claim can be considered independently supported; this is noted as a named limitation in Section 6.4.

3.4.2. LinUCB Algorithm (Adapted for AETG)

```
class LinUCBAgent:
    def __init__(self, n_arms, context_dim, alpha=1.0):
        self.n_arms = n_arms
```

```
        self.d = context_dim
        self.alpha = alpha
        self.A = [np.eye(self.d) for _ in range(n_arms)]
        self.b = [np.zeros(self.d) for _ in range(n_arms)]
        self.theta = [np.zeros(self.d) for _ in range(n_arms)]

    def select_arm(self, context):
        ucb_values = []
        for a in range(self.n_arms):
            theta_a = np.linalg.solve(self.A[a], self.b[a])
            self.theta[a] = theta_a
            ucb = context @ theta_a + self.alpha * np.sqrt(context @ np.
                linalg.inv(self.A[a]) @ context)
            ucb_values.append(ucb)
        return int(np.argmax(ucb_values))

    def update(self, arm, context, reward):
        self.A[arm] += np.outer(context, context)
        self.b[arm] += reward * context
```

3.4.3. Redis Feedback Loop Architecture

The Redis feedback loop operates as follows:

- Suricata emits Eve JSON alerts to Redis channel `ids/alerts`.
- A Feedback Worker (Python daemon) subscribes to the channel, parses alerts, matches them to flow IDs in a registry, computes reward using alert rate and ESM, and pushes the reward to the MAB agent.
- The MAB agent (LinUCB) updates its parameters based on the received reward.

Eve JSON Alert Structure (relevant fields):

```
{
```

```
  "timestamp": "2026-08-19T10:30:45.123Z",
  "event_type": "alert",
  "flow_id": "192.168.1.10:54321-10.0.0.5:443",
  "alert": {
    "signature_id": 2024567,
    "signature": "ET MALWARE Cobalt Strike Malleable C2 Profile",
    "category": "Malware Command and Control",
    "severity": 1
  },
  "tls": {
    "ja3": "771,4865-4866-4867...",
    "ja3s": "771,4865-4866-4867..."
  }
}
```

Feedback Worker Processing:

```
def process_alert(alert_msg, flow_registry):
    flow_id = alert_msg['flow_id']
    if flow_id not in flow_registry:
        return None
    flow_meta = flow_registry[flow_id]
    strategy_used = flow_meta['evasion_strategy']
    esm = flow_meta['esm']
    alert_rate = 1.0
    reward = (1 - alert_rate) * (1 - esm) # = 0
    return strategy_used, flow_meta['context'], reward
```

Non-Alert Flows: Periodic heartbeat computes reward for flows with no alerts:

```
def compute_reward_no_alert(flow_meta):
    alert_rate = 0.0
    esm = flow_meta['esm']
    return (1 - alert_rate) * (1 - esm) # = 1 - esm
```

3.4.4. Handling Delayed/Partial Feedback

Real-world IDS feedback has latency (seconds to minutes). AETG handles this via:

1. **Flow Registry**: In-memory map of `flow_id` $\rightarrow$ {context, strategy, timestamp, esm}
2. **Pending Queue**: Flows awaiting feedback
3. **Timeout Handling**: If no alert after $T_{\max} = 30$s, treat as "no alert" (reward $= 1 - \text{ESM}$)
4. **Batch Updates**: LinUCB updated in mini-batches every 5s for efficiency

3.5. Evasion Stealth Metric (ESM) — Detailed Formulation

3.5.1. Feature Extraction

For each flow, extract feature vector $\phi(f) \in \mathbb{R}^m$:

Feature Group	**Features (examples)**	**Dim**
Flow Volume	total_bytes, total_pkts, duration, bytes_per_pkt, pkts_per_sec	5
Directionality	fwd_bytes, bwd_bytes, fwd_pkts, bwd_pkts, ratio_fwd_bwd	5
TCP Flags	syn_cnt, ack_cnt, fin_cnt, rst_cnt, psh_cnt, urg_cnt, flag_entropy	7
Inter-Arrival	iat_mean, iat_std, iat_min, iat_max, iat_p25, iat_p50, iat_p75	7
Packet Size	psize_mean, psize_std, psize_min, psize_max, psize_entropy	5
TLS	ja3_entropy, ja4_entropy, cipher_diversity, ext_diversity, version	5
Total		**34**

3.5.2. Distribution Estimation

Given baseline flows $\mathcal{B} = \{b_1, \ldots, b_N\}$ and adversarial flows $\mathcal{A} = \{a_1, \ldots, a_M\}$:

1. Discretize each feature dimension into K bins using quantile binning on $\mathcal{B} \cup \mathcal{A}$.
2. Compute histograms $P_i(k)$ for baseline, $Q_i(k)$ for adversarial, per feature i.
3. Per-feature JSD:

$$\mathrm{JSD}_i = \mathrm{JSD}(P_i \parallel Q_i) = \frac{1}{2} \sum_{k=1}^{K} \left[P_i(k) \log \frac{2P_i(k)}{P_i(k) + Q_i(k)} + Q_i(k) \log \frac{2Q_i(k)}{P_i(k) + Q_i(k)} \right].$$

4. Aggregate ESM (weighted average):

$$\mathrm{ESM} = \frac{\sum_{i=1}^{m} w_i \cdot \mathrm{JSD}_i}{\sum_{i=1}^{m} w_i},$$

where weights w_i reflect feature importance (default: uniform $w_i = 1$).

3.5.3. Normalization

$\mathrm{JSD} \in [0, \ln 2]$. Normalize to $[0, 1]$:

$$\mathrm{ESM}_{\mathrm{norm}} = \frac{\mathrm{ESM}}{\ln 2}.$$

Interpretation:

- $\mathrm{ESM}_{\mathrm{norm}} \approx 0$: Adversarial traffic statistically indistinguishable from baseline (high stealth)
- $\mathrm{ESM}_{\mathrm{norm}} \approx 1$: Maximally divergent (low stealth, easily detectable by anomaly-based IDS)

3.5.4. Online ESM Computation

For real-time MAB reward, ESM is computed incrementally using exponential moving average of per-feature histograms:

$$H_i^{(t)}(k) = (1 - \beta) H_i^{(t-1)}(k) + \beta \cdot \mathbb{I}[\mathrm{bin}(\phi_i(f_t)) = k],$$

with $\beta = 0.01$ (slow adaptation). JSD computed between current $H_i^{(t)}$ and frozen baseline P_i.

3.6. Threat Model

AETG operates under the following threat model assumptions:

1. **Adversary Knowledge**: Gray-box — aware of IDS deployment (Suricata/Zeek) and public rule sets, but not internal ML weights.
2. **Capability**: Inject traffic and observe high-level alerts via Redis.
3. **Stealth Constraint**: Traffic must be protocol-compliant and statistically resemble benign traffic (quantified by ESM).
4. **Operational Context**: Single compromised host within network.
5. **Defender Response**: IDS rules may update; AETG adapts via continuous feedback.

3.7. Limitations

1. **Scalability**: CPU-only, tested on mid-range laptop; high-throughput requires GPU/distributed deployment.
2. **Attack Types**: Currently 6 attack types; extensible but needs domain expertise.
3. **Stealth Metric**: Relies on static benign baseline; does not handle concept drift.
4. **Generalization**: Validated on Suricata/Zeek; may not generalize to other IDS engines.
5. **Feedback Latency**: 30s timeout may be insufficient in high-volume environments.
6. **Ethical Constraints**: Designed exclusively for red-team validation in controlled labs.

3.8. Experimental Setup

3.8.1. Mini-SOC Enterprise Architecture

The validation environment replicates a realistic enterprise network with the following components:

- Attacker subnet: 10.10.0.0/16 (AETG node, Kali tools)

- Victim subnet: 192.168.10.0/24 (Web: .10, DB: .20, DNS: .30, AD: .40)
- SOC subnet: 172.16.0.0/16 (Suricata: .10, Zeek: .20, Elastic: .30-.32)
- SPAN/TAP: Traffic mirrored to Suricata + Zeek

The mini-SOC includes Suricata IDS with Emerging Threats rules, Zeek network analysis, and Elastic SIEM with Kibana.

3.8.2. Dataset and Traffic Generation

Dataset	**Purpose**	**Size**
CIC-IDS2017 [12]	Baseline benign traffic profiles	2.8M flows
CSE-CIC-IDS2018	Additional benign + attack variety	16M flows
Custom TLS corpus	JA3/J4 baseline for legitimate clients	50K Client Hellos

Attack Types Tested:

1. HTTP Flood (Layer 7 DDoS)
2. SSH Brute Force (Credential stuffing)
3. DNS Exfiltration (Data tunneling via TXT records)
4. C2 Beaconing (Cobalt Strike / Sliver profiles)
5. SMB Lateral Movement (PsExec, WMI)
6. Custom Payload (User-defined Scapy templates)

3.8.3. Evaluation Metrics

Metric	**Definition**	**Target**
Evasion Rate (ER)	$\frac{\text{flows without alert}}{\text{total flows}}$	$> 80\%$
Mean ESM	Average normalized JSD across flows	< 0.3
Adaptation Speed	Rounds to reach 90% of max ER	< 50 rounds
JA3 Diversity	Unique JA3s / total flows	> 0.9
False Positive Impact*	Benign flow alert rate during test	$< 2\%$

*Not measured in this work; see Section 5, "False Positive Impact".

3.8.4. Baseline Comparisons

Baseline	**Description**
Static FGSM	Feature-space PGD on flow features (protocol-invalid)
Random Evasion	Uniform random strategy selection
Fixed Strategy	Best single strategy from grid search
EXP3 Bandit	Adversarial bandit (no context)
AETG (Ours)	Full framework: LinUCB + JA3/J4 + ESM reward

3.8.5. Hardware and Software Constraints

Resource	**Specification**
CPU	AMD Ryzen 7 5800H (8C/16T)
RAM	14 GB DDR4
Storage	512 GB NVMe
GPU	None (CPU-only)
OS	Ubuntu 26.04 LTS
Python	3.11+
Key Dependencies	scapy, numpy, scikit-learn, redis, cryptography, pandas

All experiments run locally without cloud GPUs, consistent with constrained cybersecurity research principles.

4. Implementation

4.1. System Architecture

AETG is implemented as a modular Python 3.11+ framework with six core components, each designed for independent testing and extension. The system is structured to support both interactive command-line usage and automated batch experimentation, with Redis as the backbone for real-time feedback.

The codebase is organized as follows:

```
adversarial-traffic-generator/
 new_traffic_gen.py # CLI entry point
 requirements.txt # Python dependencies
 src/
    __init__.py
    traffic_gen.py # Core traffic generation (HTTP, DNS, SSH)
    mab_optimizer.py # LinUCB implementation
    stealth_metric.py # ESM (Jensen-Shannon divergence)
 experiments/
    adv_training.py # Adversarial training script
    eval_ids.py # IDS evaluation script
 data/
    malicious_templates/ # PCAP templates (placeholder)
    results/ # Experiment logs and summaries
    features/ # Feature datasets
 notebooks/ # Jupyter notebooks for analysis
 tests/ # Unit tests
```

4.2. Core Modules

4.2.1. Traffic Generator (src/traffic_gen.py)

The traffic generator module provides three primary functions: HTTP/HTTPS request generation with JA3/J4 fingerprinting, DNS query generation (simulated wire format), and SSH connection simulation. All requests include configurable timing jitter to mimic legitimate traffic patterns.

4.2.2. Multi-Armed Bandit (src/mab_optimizer.py)

The `ContextualMAB` class implements LinUCB with ϵ-greedy exploration. The implementation uses ridge regression with design matrix A_a and reward vector b_a. The exploration parameter $\epsilon = 0.1$ ensures 10% random arm selection.

4.2.3. Stealth Metric (src/stealth_metric.py)

The ESM evaluator computes Jensen-Shannon Divergence between benign and adversarial feature distributions. The function accepts feature dictionaries, computes

histograms over 10 bins per feature, and returns a normalized stealth score.

4.2.4. CLI Entry Point (`new_traffic_gen.py`)

The CLI orchestrates the entire pipeline. It supports flags for URL, JA3/J4 specification, number of requests, adaptive mode, context vector, Redis feedback, and verbose logging. The integration flow involves parsing arguments, initializing components, and for each request: observing context, selecting strategy via MAB, generating traffic, computing reward, and updating MAB.

4.3. Redis Feedback Loop Integration

The Redis feedback loop is implemented as a pub/sub pattern:

- Suricata → Eve JSON alerts → Redis channel `ids/alerts`
- Feedback Worker (Python daemon) subscribes, parses alerts, matches to flow registry, computes reward, and pushes to MAB queue.
- MAB Agent consumes queue and updates LinUCB parameters.

Delayed feedback is handled via flow registry with timeout ($T_{\max} = 30$s) and batch updates every 5 seconds.

4.4. Integration with mini-SOC

The mini-SOC enterprise architecture is deployed via Docker Compose with services for Suricata, Zeek, Elasticsearch, and Kibana. AETG communicates with Redis and sends traffic to victim hosts, mirrored to Suricata and Zeek via SPAN/TAP.

4.5. Engineering Challenges and Solutions

4.6. Deployment and Execution

Prerequisites: install dependencies, start Redis, optionally start mini-SOC. Run experiments via the CLI.

Challenge	Solution
CPU-only performance	Vectorized NumPy; batch updates; lightweight ML
Feedback latency	Asynchronous Redis pub/sub with timeout
Protocol compliance	Scapy packet crafting; tls_client for valid TLS
Reproducibility	Seeded random; JSON logging with timestamps
Memory constraints (14GB)	In-memory flow registry with LRU eviction

5. Experimental Results

5.1. Experimental Setup

All experiments were conducted on a commodity laptop (AMD Ryzen 7 5800H, 14GB DDR4, no GPU) running Ubuntu 26.04 LTS. Redis server and a mock HTTP server (`python3 -m http.server 8082`) were co-located. The mini-SOC was deployed in a separate virtual network segment.

Datasets: benign baseline from CIC-IDS2017 (1000 flows of legitimate HTTPS), adversarial templates for 6 attack types (focus on HTTPS C2 beaconing).

Metrics: Evasion Rate (ER), ESM, Strategy Diversity, Adaptation Speed.

5.2. Evasion Rate: Random vs. LinUCB (Calibration Environment)

The experiment below was run against a mock HTTP server with no active IDS rules, not the mini-SOC (Suricata with Emerging Threats rules) described in Section 3.8.1. It functions as a pipeline/calibration test, not an IDS-evasion validation. The 96.8% figure carried into the Abstract originates from this experiment; the corresponding evaluation against the mini-SOC and a trained ML detector is reported separately in Section 5.9. The random baseline achieved 100% evasion because all requests used randomly selected JA3 fingerprints against a mock HTTP server with no IDS rules. LinUCB achieved 96.8% while exploring 5 distinct evasion strategies, demonstrating effective balance between exploration and exploitation.

The "Random (Baseline)" evasion rate reported here (100%) differs from the "Random Evasion" row in Table 5.6 (52%) because the two rows reflect different experimental conditions: the 100% figure above is measured against the mock HTTP server

Experiment	Total Requests	Successful Requests	Evasion Rate	Unique Strategies
Random (Baseline)	1000	1000	100%	1
LinUCB (Adaptive)	1000	968	96.8%	5

used for calibration, while Table 5.6 (Section 5.6) reports a separate calibration-suite run in which the random-strategy baseline faced a wider mix of traffic templates. Both remain mock-server, non-mini-SOC results; neither has been re-run against Suricata.

5.3. Strategy Distribution

Strategy	Selection Count	Percentage
Baseline (no evasion)	42	4.2%
Fragmentation	58	5.8%
Timing Jitter	76	7.6%
Padding	34	3.4%
TLS Cipher Shuffle	312	31.2%
TLS Extension Shuffle	289	28.9%
Combined (Frag+Time)	98	9.8%
Combined (TLS+Pad)	91	9.1%

TLS-based strategies dominated (>60%), confirming effectiveness against rule-based IDS.

5.4. Adaptation Speed

LinUCB reached 90% of its maximum evasion rate within 38 rounds, significantly faster than EXP3 (62 rounds). The learning curve shows initial fluctuation due to exploration ($\epsilon = 0.1$) and convergence by round 40.

Strategy	Mean ESM (normalized)
Baseline (no evasion)	0.21
Fragmentation	0.28
Timing Jitter	0.19
Padding	0.22
TLS Cipher Shuffle	0.09
TLS Extension Shuffle	0.11
Combined (Frag+Time)	0.24
Combined (TLS+Pad)	0.15

5.5. Stealth Analysis (ESM)

TLS-based strategies achieved the lowest ESM (0.09–0.11), preserving TLS handshake structure. Overall average ESM = 0.14, well below the target of 0.3.

5.6. Comparison with Baselines (Calibration Environment)

Method	Evasion Rate	Mean ESM	Adaptation Speed	Strategy Diversity
Static FGSM	34% (invalid packets)	0.45	N/A	1
Random Evasion (uniform)	52%	0.28	N/A	8
Fixed Strategy (TLS Cipher Shuffle)	83%	0.09	N/A	1
EXP3 Bandit (no context)	89%	0.22	62 rounds	6
AETG (LinUCB)	**96.8%**[1]	**0.14**[1]	**38 rounds**[1]	**5**

[1] All figures in this table, including AETG's, were obtained in the calibration environment (mock HTTP server, no active IDS rules) described in Sections 5.1–5.2. They

should be read as a comparison of bandit/strategy behavior under identical calibration conditions, not as a comparison of evasion effectiveness against a production IDS. Against the mini-SOC deployment with a trained ML detector (Section 5.9), the evasion rate for AETG's generated traffic was 0.0% (detector recall = 1.0) on the tested set of 200 adversarial flows. The alert transport layer was confirmed functional in separate validation; the reported zero alert count for that 200-flow batch was traced to a key-pattern mismatch in the evaluation script (see Section 5.9), not to an absence of Suricata-generated alerts. This baseline comparison table has not yet been re-run under mini-SOC conditions for the other methods, so no like-for-like comparison against a production IDS is currently available for any row.

Within the calibration environment, AETG's LinUCB agent outperforms the other tested baselines in evasion rate, ESM, and adaptation speed; this ranking has not been established against the mini-SOC.

5.7. Statistical Significance

Paired t-tests over 10 independent runs show:

- AETG vs. EXP3 (Evasion Rate): p = 0.012 (significant)
- AETG vs. EXP3 (Mean ESM): p = 0.008 (highly significant)
- AETG vs. Fixed Strategy (Evasion Rate): p = 0.031 (significant)

These significance tests were computed within the calibration environment only and have not been repeated against the mini-SOC.

5.8. False Positive Impact

Section 3.8.3 defines "False Positive Impact" (target: benign flow alert rate $< 2\%$ during test) as one of five core evaluation metrics. This metric was not measured in this work; no benign-flow alert rate was recorded during either the calibration-environment runs or the mini-SOC deployment. It is retained in Table 3.8.3 (marked accordingly) as an open item for future work rather than removed, since it remains part of the intended evaluation design.

5.9. Mini-SOC Integration Status

The results reported in Sections 5.2–5.7 were obtained against a mock HTTP server without active IDS rules and should be read as calibration/pipeline validation, not IDS-evasion validation. This subsection reports the status of the mini-SOC deployment described in Section 3.8.1.

Deployment. The mini-SOC was deployed via Docker Compose with Redis, Suricata, an alerting service, a dashboard, and a mock endpoint; all containers started successfully.

Rule loading. Suricata loaded 52,415 rules from the Emerging Threats rule set.

ML detector evaluation. An XGBoost classifier was evaluated against 200 adversarial C2 beacon flows generated by AETG (alongside 200 benign flows). The classifier achieved recall = 1.0, precision = 0.5, F1 = 0.667, AUC = 0.5025, and evasion_rate = 0.0, meaning none of the generated adversarial flows evaded detection. This result stands in direct contrast to the 96.8% evasion rate reported in Section 5.2. The precision of 0.5 and AUC near 0.5 indicate the classifier is currently over-flagging (low specificity) even as it catches every adversarial flow; this is itself a limitation of the current ML-detector configuration, not a claim of a well-calibrated production detector.

Alert pipeline validation. In separate deployment smoke-tests, the alert transport layer (Suricata → `eve.json` → Redis) was confirmed functional: Suricata generated alerts and `alert:*` keys were observed in Redis. For the 200-flow batch evaluated against the ML detector, the evaluation script (`eval_ids.py`) reported `alert_count_in_redis = 0`. This was traced to a key-pattern mismatch between the evaluation and ingestion scripts: `eval_ids.py` queries `r.llen('alerts')` (a Redis list), whereas `push_alerts.py` stores each alert as an individual key via `r.set('alert:*', ...)`. The two scripts are therefore reading and writing different key structures, so a count of zero from `r.llen('alerts')` does not indicate that Suricata failed to generate alerts for this batch — it reflects a bug in how the evaluation script queries Redis. The alert pipeline itself was verified operational in the separate smoke-test described above; whether alerts

were in fact generated for this specific 200-flow batch has not been confirmed by inspecting `alert:*` keys directly, and doing so is a prerequisite for closing the loop below.

Feedback loop completeness. While the transport layer (Suricata → Redis) was confirmed functional in isolation, the closed loop in which the LinUCB agent consumes these alerts as its live reward signal (as specified in Sections 3.4.3–3.4.4) was not exercised end-to-end in this session. The LinUCB results in Sections 5.2–5.7 were obtained using the mock-server reward signal, not Suricata- or ML-derived alerts.

Interpretation. Together, these results indicate that AETG's traffic-generation and infrastructure components function as designed, but the framework's central evasion claim has not yet been tested under the conditions it is meant to validate. The gap between the 96.8% and 0.0% evasion figures is itself informative: it suggests the mock-server calibration condition substantially underestimates detection difficulty relative to a trained ML-based detector, even one with the calibration issues noted above. Fixing the evaluation script's key-pattern bug, confirming alert counts directly against `alert:*` keys for the 200-flow batch, and completing the closed feedback loop so LinUCB trains on Suricata/ML-derived rewards are the necessary next steps before any evasion claim against a production-grade IDS can be made.

5.10. Limitations of Results

The results have limitations: mock IDS environment used for the headline calibration figures, ESM baseline dependency, limited attack types, CPU-only constraints, an unresolved key-pattern mismatch in the alert-count evaluation script, and — as detailed in Section 5.9 — an as-yet-incomplete closed feedback loop against the deployed mini-SOC. Future work should address these.

6. Discussion

6.1. Interpretation of Results

Within the controlled calibration environment, the 96.8% evasion rate, mean ESM of 0.14, and adaptation within 38 rounds collectively demonstrate that AETG's traffic-generation and bandit-selection pipeline functions as designed. TLS-based strategies achieved the lowest ESM among the tested strategies, consistent with preserving TLS handshake structure. However, as reported in Section 5.9, this pipeline has not yet been exercised in closed loop against the mini-SOC, and a preliminary check against a trained ML classifier found no evasion (recall = 1.0) on adversarial C2 flows. The exploration–exploitation trade-off observed in the calibration environment is a necessary but not sufficient condition for adaptability against a real detector.

6.2. Implications for IDS Deployment

- Signature-based IDS alone is insufficient; anomaly-based ML models should be integrated.
- JA3/J4 whitelisting is not a panacea; adversaries can generate unlimited valid fingerprints.
- Regular red-team testing using frameworks like AETG can proactively identify weaknesses.

Defensive strategies include moving target defense, feature engineering for distributional awareness, and ensemble methods.

6.3. Comparison with State-of-the-Art

Within the calibration environment, AETG's measured evasion rate, ESM, and adaptation speed compare favorably to the baselines summarized in Table 5.6. A direct comparison with prior works (Anderson et al., Wang et al., DeepRed) against a shared, production-grade IDS has not been performed. AETG's intended differentiators — protocol-aware generation, contextual MAB,

ESM-guided reward, and real-time feedback — remain the paper's design contribution; their empirical advantage over prior work against a live IDS is a claim for future validation, not a result established here.

6.4. Limitations

Experimental constraints (mock environment, limited attack types, CPU-only) and methodological limitations (ESM baseline dependency, reward formulation assumptions, LinUCB linearity assumptions) are acknowledged. Ethical constraints are inherent to offensive security research.

6.5. Future Work

Technical extensions: multi-IDS evasion, transfer learning, adversarial training, real-time rule adaptation. Research directions: theoretical guarantees, feature selection, human-in-the-loop, real-world validation. Broader impact includes botnet detection, automated red-teaming, and security education.

7. Conclusion

7.1. Summary of Contributions

This paper presented AETG, a framework that:

1. Generates protocol-compliant adversarial traffic with TLS fingerprint randomization.
2. Implements a LinUCB-based contextual bandit for adaptive evasion strategy selection.
3. Introduces ESM as a candidate quantitative stealth metric based on Jensen-Shannon Divergence.
4. Was deployed against a mini-SOC environment (Suricata with 52,415 Emerging Threats rules, Redis-based alert pipeline), successfully validating the infrastructure components.

7.2. Key Findings

1. In a calibration environment (mock HTTP server, no active IDS rules), the LinUCB agent achieved a 96.8% evasion rate, mean ESM of 0.14, and adaptation within 38 rounds — outperforming the tested baselines under identical calibration conditions.
2. The mini-SOC deployment's infrastructure (containers, Suricata rule loading) was confirmed functional.
3. Against a trained ML detector (XGBoost) evaluated on 200 adversarial C2 beacon flows, evasion rate was 0.0% (recall = 1.0), directly contrasting with the calibration-stage evasion figure and indicating that the tested evasion strategies do not transfer to this ML detector. The detector's precision (0.5) and AUC (≈0.50) indicate it is currently miscalibrated (over-flagging), so this result should be read as "not evaded" rather than as validation of a production-quality detector.
4. The alert transport layer (Suricata → `eve.json` → Redis) was confirmed functional in separate validation, with `alert:*` keys observed in Redis. The zero alert count reported for the 200-flow ML evaluation batch was traced to a key-pattern mismatch between the evaluation script (`r.llen('alerts')`) and the ingestion script (`r.set('alert:*', ...)`), not to a failure of Suricata to generate alerts.
5. The full closed-loop integration — the LinUCB agent training on live Suricata/ML-derived rewards rather than the mock-server proxy — was not completed, and False Positive Impact was not measured.

7.3. Practical Implications

The results to date support a narrower claim than the framework's design goal: AETG's traffic-generation and infrastructure pipeline are functional and can be deployed against a realistic mini-SOC stack, but the framework has not yet demonstrated evasion of that stack — and the one direct test performed (against the XGBoost detector) showed the opposite result. For practitioners, this is itself a useful finding: it suggests that flow-level ML detection, even in

an under-calibrated configuration, can be robust to the TLS-fingerprint- and timing-based evasion strategies explored here. For academics building on this work, the immediate priorities are (1) fixing the evaluation script's key-pattern bug and confirming alert counts directly against `alert:*` keys, and (2) closing the feedback loop so LinUCB is trained against live IDS/ML signals rather than a mock-server proxy, after which the evasion, stealth, and adaptation-speed claims can be re-evaluated under conditions that actually test them.

7.4. Closing Remarks

AETG's architecture — protocol-aware generation, contextual bandit selection, and an information-theoretic stealth metric — remains a reasonable design for red-team IDS validation, and its supporting infrastructure (mini-SOC, Suricata, Redis alert pipeline) is deployed and partially verified. However, the current results validate the pipeline's mechanics, not its central evasion claim: the only direct test against a trained detector showed complete detection. Reporting this gap explicitly, rather than the calibration-stage 96.8% figure alone, is necessary for the paper to accurately represent what has been demonstrated versus what remains future work.

Acknowledgments

The author would like to thank the Institut Teknologi dan Bisnis STIKOM Bali for supporting this research.

References

[1] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*, 2015.

[2] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, 2017.

[3] Hyrum S. Anderson, Anant Kharkar, Bobby Filar, David Evans, and Phil Roth. Learning to evade static pe machine learning malware models via reinforcement learning. In *Black Hat USA*, 2017.

[4] Giovanni Apruzzese, Michele Colajanni, Luca Ferretti, Alessandro Guido, and Mirco Marchetti. The role of machine learning in cybersecurity: A survey. *ACM Computing Surveys*, 54(1):1–38, 2021.

[5] Yixin Hao, Ming Li, Xin Hu, and Xin Wang. Moving target defense via multi-armed bandit. *IEEE Transactions on Dependable and Secure Computing*, 17(3):493–506, 2020.

[6] Mehrdad Hajizadeh, Pegah Golchin, Ehsan Nowroozi, Maria Rigaki, Veronica Valeros, Sebastián García, Mauro Conti, and Thomas Bauschert. DeepRed: A deep learning-powered command and control framework for multi-stage red teaming against ML-based network intrusion detection systems. In *Proceedings of the 19th USENIX Workshop on Offensive Technologies (WOOT)*, 2025.

[7] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*, 2018.

[8] Yue Wang, Zheng Zhang, Xin Liu, and Xiang Li. Reinforcement learning for adaptive adversarial traffic generation. In *Network and Distributed System Security Symposium (NDSS)*, 2022.

[9] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *International World Wide Web Conference (WWW)*, 2010.

[10] Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. The nonstochastic multiarmed bandit problem. *SIAM Journal on Computing*, 32(1):48–77, 2002.

[11] Yuan Zhuang, Xin Liu, Zheng Zhang, and Ke Xu. Adaptive honeypot engagement via adversarial bandits. In *International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2021.

[12] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset. In *International Conference on Security and Privacy in Communication Networks*, 2018.